

Prácticas de Ampliación de Sistemas Operativos •

Tarea entregable: exec_lines

Descripción del programa `exec_lines`

Escribe un programa llamado `exec_lines` que lea un flujo de líneas (secuencias de bytes terminadas en `\n`) por la entrada estándar en **bloques de tamaño configurable**, y ejecute cada una de dichas líneas como si fueran órdenes introducidas por teclado. Para simplificar el problema, supondremos que el único carácter de control que se recibe es `\n`, y que el único separador es el espacio en blanco (no hay tabuladores). También supondremos que **las líneas no tendrán más de un tamaño máximo determinado**, y si una línea fuera mayor, devolveríamos un error indicándolo, y terminaríamos la ejecución. Cada una de las líneas a ejecutar puede tener **una** redirección o **una** tubería, es decir, que cada línea puede ser de los siguientes cinco tipos:

- `comando opciones`
- `comando opciones < fichero_entrada`
- `comando opciones > fichero_salida`
- `comando opciones >> fichero_salida`
- `comando1 opciones1 | comando2 opciones2`

donde `comando` es cualquier ejecutable. Ejemplos serían:

- `cat mi_fichero`
- `wc -l < fichero_entrada`
- `ls -la > fichero_salida`
- `df -h -u >> fichero_salida`
- `ls -l -a | wc -l`

El programa tiene que aceptar las siguientes opciones:

```
./exec_lines [-b BUF_SIZE] [-l MAX_LINE_SIZE] [-p NUM_PROCS]
```

donde `BUF_SIZE` tomará el valor 16 por defecto, siendo el tamaño de buffer que se deberá usar al leer la entrada estándar, y `MAX_LINE_SIZE` es el tamaño máximo de la línea que admitimos, tendrá un valor por defecto de 32 caracteres y **si una línea supera esa longitud, `exec_lines` sacará un error indicándolo** (el texto del error se puede ver en los ejemplos de ejecución). Finalmente, `NUM_PROCS` tomará el valor 1 por defecto y es el número de líneas del flujo de entrada que se estarán ejecutando en paralelo en cada momento. Representa por tanto, el número máximo de comandos concurrentes "al vuelo". De esta manera, si `NUM_PROCS` es 1, cada línea del fichero se ejecutará secuencialmente en un proceso aparte, esperando el padre a que acabe antes de ejecutar la siguiente. Si `NUM_PROCS` fuera N, se lanzarán N comandos, uno por cada una de las N primeras líneas de la entrada (si es que hay 2 o más líneas) y se irán ejecutando nuevas líneas conforme vayan terminando los ya existentes, de manera que siempre haya N comandos en ejecución. Nótese que en caso de un comando con tubería, se lanzarán dos procesos que contarán como un único comando en el paralelismo.

Explicación detallada

El programa deberá leer la entrada estándar línea a línea usando la llamada al sistema `read` con el tamaño de bloque que se le pase. Si la línea supera la longitud máxima permitida no se ejecutará, sino que se sacará un mensaje de error indicando el número de línea y su contenido, y se parará la ejecución.

Una vez leída la línea, tendréis que analizarla buscando alguno de los operadores de redirección o tubería: `<`, `>`, `>>`, `|`. **Solamente dejamos que aparezca uno de ellos**, por lo que si hay más de uno, deberéis sacar un mensaje de error (ver los ejemplos de ejecución). En el caso de los operadores de redirección, vuestro programa tendrá que abrir el correspondiente fichero (para lectura o escritura según corresponda, añadiendo o sobrescribiendo según el separador), hacer la adecuada redirección en la tabla de descriptores y ejecutar el programa solicitado. En el caso del operador de tubería, tendréis que lanzar dos procesos: el de la parte izquierda con la redirección adecuada para escribir en la tubería, y el de la parte derecha con la redirección para leer de la tubería, ejecutando cada uno el programa adecuado.

El programa desarrollado en esta tarea debe maximizar el paralelismo, es decir, deberá ejecutar `NUM_PROCS` comandos de forma simultánea si hay suficientes líneas para ello. Si el número de comandos excede `NUM_PROCS` deberá esperar a que termine alguno de los comandos que están en ejecución. La gestión de este procedimiento se debe hacer usando señales de forma similar a como se ha visto en la última sesión de prácticas de llamadas al sistema para detectar la finalización de los procesos que ejecutan los comandos.

En el caso de que la ejecución de alguna línea produjera un valor de salida distinto de 0, se detendría la ejecución de `exec_lines` de forma ordenada, es decir, se esperará a que terminen correctamente (o no) el resto de procesos paralelos antes de finalizar la ejecución principal. Para informar del problema, se sacará en la salida estándar de error el número de línea que causó el error y el valor de exit del proceso convenientemente interpretado. Si fue un valor de terminación por un exit indicando un error, se mostrará el valor devuelto, y si la terminación se produjo por una señal que mató al proceso, se mostrará el valor de dicha señal.

Ejemplos de ejecución de `exec_lines`

```
$ ./exec_lines -h
Uso: ./exec_lines [-b BUF_SIZE] [-l MAX_LINE_SIZE] [-p NUM_PROCS]
Lee de la entrada estándar una secuencia de líneas conteniendo órdenes
para ser ejecutadas y lanza los procesos necesarios para ejecutar cada
línea, esperando a su terminación para ejecutar la siguiente.
-b BUF_SIZE          Tamaño del buffer de entrada 1<=BUF_SIZE<=8192
-l MAX_LINE_SIZE     Tamaño máximo de línea 16<=MAX_LINE_SIZE<=1024
-p NUM_PROCS         Número de procesos en ejecución de forma simultánea (1 <=
NUM_PROCS <= 8)

$ echo $?
0

$ ./exec_lines -b
./exec_lines: option requires an argument -- 'b'
Uso: ./exec_lines [-b BUF_SIZE] [-l MAX_LINE_SIZE] [-p NUM_PROCS]
Lee de la entrada estándar una secuencia de líneas conteniendo órdenes
para ser ejecutadas y lanza los procesos necesarios para ejecutar cada
```

```
línea, esperando a su terminación para ejecutar la siguiente.
-b BUF_SIZE          Tamaño del buffer de entrada 1<=BUF_SIZE<=8192
-l MAX_LINE_SIZE     Tamaño máximo de línea 16<=MAX_LINE_SIZE<=1024
-p NUM_PROCS         Número de procesos en ejecución de forma simultánea (1 <=
NUM_PROCS <= 8)

$ echo $?
1

$ ./exec_lines -b 0
Error: El tamaño de buffer tiene que estar entre 1 y 8192.

$ echo $?
1

$ ./exec_lines -p 0
Error: El número de procesos en ejecución tiene que estar entre 1 y 8.

$ echo $?
1

$ /genera_bytes.py -n 129 | base64 | tr -d '\n' | ./exec_lines -l 128,
Error, línea 1 demasiado larga: "nXmxo38xgBzRGmcG+0DWvVdSaEaQ07E+3lYkOenBu
COpYIm8px89Gm0tPK2zZpy9UOF15DQknYuCn0EWaYQql5kRA2zz6CIIbsqgB1pp/BeLqPg3GKq
P09H2XoFE5h2a..."

$ echo $?
1

$ touch test ; echo ls test | ./exec_lines
test

$ echo $?
0

$ echo -e "ls /home/usuario/directorio/muylargo/para/generar/una/linea/enorme" |
./exec_lines -l 16
Error, línea 1 demasiado larga: "ls /home/usuario..."

$

$ touch test ; echo -e "ls test\nls test" | ./exec_lines
test
test

$ touch test ; echo -e "ls test\nls notest" | ./exec_lines
test
Error al ejecutar la línea 2. Terminación normal con código 2.

$ echo -e "ls\nevince" | ./exec_lines # matamos evince con un kill desde otro
terminal
test
Error al ejecutar la línea 2. Terminación anormal por señal 9.
```

```
$ touch test ; echo -e "ls test > salida\ncat salida" | ./exec_lines
test

$ touch test ; echo -e -n "ls test >> salida\ncat salida" | ./exec_lines
test
test

$ touch test ; echo -e "ls test | wc -c" | ./exec_lines
5

$ (time -p echo -e "sleep 1\nsleep 1\nsleep 1\n" | ./exec_lines -p 1) |& grep real
| cut -d ',' -f 1
real 3

$ echo $?
0

$ (time -p echo -e "sleep 1\nsleep 1\nsleep 1\n" | ./exec_lines -p 2) |& grep real
| cut -d ',' -f 1
real 2

$ echo $?
0

$ (time -p echo -e "sleep 1\nsleep 1\nsleep 1\n" | ./exec_lines -p 3) |& grep real
| cut -d ',' -f 1
real 1

$ echo $?
0
```

Entrega

La entrega consistirá en un único archivo `exec_lines.c`. El archivo debe compilar conforme a los archivos `tasks.json` o `Makefile` incluidos en `ASO-Semana1.tar.xz`, es decir, con `gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11 exec_lines.c -o exec_lines`. La primera línea de `exec_lines.c` debe ser la directiva para compilar conforme al estándar POSIX: `#define POSIX_C_SOURCE 200809L`. El fichero `exec_lines.c` deberá pasar como mínimo los tests que se os indican en este documento.

Evaluación

Para aprobar la práctica es imprescindible que los programas pasen las pruebas de ejecución que se os proporcionarán, sin que esto signifique que no se puedan detectar otros errores con pruebas adicionales que el profesor haga durante la evaluación.

Se tendrá en cuenta la correcta indentación del código, su estructuración, el manejo de los posibles errores de las llamadas al sistema o funciones de biblioteca, así como la correcta gestión de la memoria dinámica. Otro punto importante, y por eso lo volvemos a recordar, es que se penalizará severamente el no usar buffers para las lecturas/escrituras, es decir, leer o escribir byte a byte, pudiendo llegar a suspender la práctica, así como no contemplar lecturas o escrituras parciales si fuera necesario para la garantizar la corrección del algoritmo. Adicionalmente, las ineficiencias que se detecten en el código también pueden ser objeto de penalizaciones.